

D5 — Part 1 Retrospective (Berk Hakan Öge, 210104004132)

Project: Recipix v4.1 — recipe DSL with dimensional types **My role in Part 1:** parser owner; PM-style orchestration of the developer and tester agents.

What is working

The parser landed cleanly. All 48 parser tests are green, every Part 1 spec decision (#7, #17, #20, #23, #34, #35, §7–§11) is enforced at the right point in the recursive-descent stack, and the AST shapes match spec §14 exactly — `QuantityOf` is an operator-form node, `SubstituteCall` carries `str` slots, and `QuantityLit` keeps unit and number separate. The orchestration model that I ended up using — one PM session in Claude Code (Opus 4.7) producing prompts for two separate autonomous agents (developer in Sonnet 4.6, tester in Sonnet 4.6) — worked better than I expected: the tester agent surfaced eight real parser bugs plus a spec gap that the developer agent then fixed in a single disciplined round, and crucially the PM kept the boundary between “parser bugs” and “lexer bugs” clean so neither agent stepped on my partner’s lexer code. Lexer/parser integration was painless because we agreed the `tokens.py` contract before either side started writing.

What is hard

The genuinely hard part of Part 1 was not the recursive descent — it was **second-order semantics** of design decisions that *sound* simple. Decision #17 (non-associative comparisons) is the clearest example: the developer agent encoded “comparisons cannot chain” as “no comparison can appear on either side of a comparison”, which over-rejected `a < b == c` even though that is supposed to parse as `(a < b) == c`. The tester agent caught it; the fix is small but the framing matters for the May 14 exam — the rule is “the *same* comparison cannot chain,” not “comparisons cannot compose.” Boundary conditions like zero-arg calls (`r()` vs `f()`), `quantity_of` operator vs call-form, and the `substitute` slot resolution rule (which is *not* lexical scope) all required explicit one-paragraph rationale rather than glanceable choices.

What I plan to fix in Part 2

Three carryovers. **First**, the parser currently produces `FunctionCall` for zero-arg calls regardless of whether the name resolves to a recipe; the type checker needs to disambiguate via the symbol table, which is fine, but the `AmbiguousCall` AST node we introduced as a critical fix in late Round 2 still has a few code paths in the pretty-printer that need polish. **Second**, `scale/substitute` are accepted in any expression position; the `evaluate`-only constraint is currently social, not enforced — Part 2 will move it into the type

checker so we can write a clean error message. **Third**, the **return** placement constraint (“single, last-statement”) is deferred semantic work — it parses freely now and the type checker will reject early returns. None of these are new design decisions; they are Part 1 / Part 2 boundary clean-ups that we flagged in the parser README as “spec ambiguities encountered” and consciously chose to defer rather than retrofit at deadline.